Empowering Users with Algorithmic Control and Transparency in Tourism Recommender Systems: A DSA-Compliant Approach

DAVID GALLOB, Technical University of Munich, Germany

Modern recommender systems that users encounter in daily life are very often "black boxes", which means that users rarely know how their recommendations are generated and they have no control over which items are recommended. In tourism, this leads to a lack of trust and user satisfaction, as users can never be sure if their recommendation is really optimal or influenced by profit maximization goals of the platform. The EU has therefore introduced the Digital Services Act (DSA), a framework which requires platforms to explain their algorithms in an understandable way and offer users a profiling-free alternative. In this paper, a web-based prototype for hotel recommendations which addresses these issues by implementing two algorithms is presented. The web-app offers detailed explanations of the underlying decision algorithms and shows, for each result, how the recommendation was computed. The two algorithms implemented are Weighted Multi-Attribute Utility Theory (MAUT), which lets users set weights on the different attributes (e.g. stars, distance to center, price) on a scale from 0 to 10, and Constraint Filtering, where the user sets hard limits to the attributes (e.g. price must be below 100 Euro per night). This paper details the system architecture as well as the mathematical foundations of the used algorithms along with the UI features explaining the algorithms to the user.

Additional Key Words and Phrases: Recommender Systems, User Control, Multi-Attribute Utility Theory, Constraint Filtering, Digital Services Act, Explainable AI, E-Tourism

1 Introduction

In day-to-day life, recommender systems are ubiquitous, from online shopping and video streaming to hotel and flight recommendation platforms [1]. Many prominent RS do not let users know how their recommendations are generated and offer no direct means of influencing the outcome [2]. Those systems are often built on complex algorithms (mainly neural networks) which process enormous amounts of user data to generate suggestions. They rarely explain their underlying logic to users. This lack of transparency can hinder trust and user satisfaction, leading to frustration and cognitive overhead. The tourism domain, where the financial expenditure and emotional investment are generally high, is particularly affected by this issue [8].

The Human-Computer Interaction field has focused research on user influence and transparency (explainability) to address these shortcomings [6, 12]. Instead of using automated profiling, interactive recommender systems give users the ability to influence the recommendation process by adjusting sliders representing attribute weights or setting hard constraints for the attributes (e.g., price or distance).

The demand for transparency is further fueled by the European Union's Digital Services Act, which governs online platforms and aims to protect users' rights. The DSA mandates that online platforms must provide users with transparent information about the main parameters of their recommender systems in an understandable language in plain text and offer options to influence them (Article 27) [3]. Article 38 further obliges very large online platforms (VLOPs) to provide users with at least one option for their recommender systems that is not based on profiling. These articles require developers to shift from black-box profiling towards transparent and user-controlled platforms.

In this paper, the design decisions, architecture, and implementation of a web-application tourism recommender system prototype which is compliant with the DSA are detailed. It addresses the transparency as well as the user influence goals by implementing two algorithms:

Author's Contact Information: David Gallob, david.gallob@tum.de, Technical University of Munich, School of Computation, Information and Technology, Munich, Germany.

2026. ACM 2476-1249/2026/6-ART1
https://doi.org/

(1) **Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT)**: This algorithm uses sliders to determine weights for hotel attributes (price, location, star rating, customer rating, and public transit access). To prevent user confusion regarding percentages summing up to 100%, the importance weights are defined on a 0 to 10 scale (where 0 means completely unimportant and 10 is the most important). The algorithm performs a normalization of the attributes and computes a weighted utility score for each hotel. Finally, the hotels are ranked according to their utility score (percentage match).
(2) **Algorithm B: Constraint-Based Filtering**: In Constraint-Based Filtering, the user defines hard limits for each attribute (e.g., maximum price or minimum rating). All hotels which do not meet the constraints are filtered out. This allows users to specify their preferences in a clear and understandable way without the need for complex mathematical calculations.

The prototype features a dynamic strategy selection landing wizard that ensures that transparency panels are not shown prematurely. Once a strategy is chosen, a dynamic banner is displayed explaining the chosen algorithm in clear words, and offers a button which when pressed shows the mathematical formula of the chosen algorithm for enhanced transparency. The calculated results also offer a collapsible mathematical breakdown (only on user intent to not overwhelm uninterested users) showing a step-by-step calculation of how the result's score was computed. This allows users to understand how the results are generated and how their inputs influence the outcome. The calculation breakdown is generated using KaTeX and renders the exact mathematical formulas and normalization equations used in the calculations, allowing users to trace how their inputs directly translate into the presented results.

2 Remainder Structure

The remainder of this paper is structured as follows: In section 3, the related work in current tourism recommender systems, MAUT, and explanation interfaces is presented. section 4 details the system's architecture and the mock dataset. section 5 shows the mathematical foundations of the two used algorithms. section 6 details the UI design and transparency features. section 7 proposes a protocol for a future user study and extensions, and section 8 concludes the paper. In the Appendix, the complete mock database used in the project, followed by the author contribution details and GenAI disclosures are attached.

3 Related Work

Recommender systems' development faces special challenges which are unique to their domain — such as recommendations for movies and music [2]. In the tourism domain, decisions require a lot of rational resources, are infrequent, and have multiple decision attributes. In contrast to low-cost consumer goods, vacation planning involves a high final risk and high emotional expectations, which also leads to a high time commitment. Users therefore take extensive information and comparisons into account and need to express specific constraints before making a final decision [8].

Traditional recommender systems in the tourism domain are mainly divided into three categories [4, 9]:

- **Collaborative filtering:** Builds on historical information of similar users, but suffers from the Cold Start Problem — New hotels without reviews and seldom travellers without historical ratings lead to wrong recommendations.
- **Content-based:** These systems match item features with the preference profiles of the user. While they get good results, they are often unable to explain their recommendations and to adapt to rapid changes in user needs (e.g., business travelers in central locations versus family budget accommodations).
- **Hybrid:** These recommender systems combine multiple approaches.

In order to tackle these limitations, multi-attribute frameworks, i.e., Multi-Attribute Utility Theory (MAUT) for decision making, have been coming to use in interactive recommender systems [10]. MAUT is a mathematical

algorithm which allows to evaluate alternatives with potentially conflicting attributes [5]. The algorithm defines a utility function for every attribute (e.g., mapping location to a utility between 0 and 1) and calculates a weighted sum. MAUT, as shown by Schmitt et al. in 2002 [10], is particularly useful in interactive recommender systems, as it takes human decision making processes into account (i.e., user-defined weights). It allows for deficiencies (e.g., high price) to be evened out by a high value in another attribute (e.g., excellent location).

Constraint-based recommender systems are non-compensatory in contrast. They do not calculate a score, but filter out items violating user-defined limits based on explicit knowledge about the items [8]. This approach is easy to understand and does not require user profiling, perfectly aligning with the EU's DSA [3].

Earlier research has shown that the success of interactive recommender systems strongly relies on their UIs [7, 11]. Therefore, providing explanations in the UI improves user understanding as well as their trust and perceived control over the system [12]. Knijnenburg et al. [6] showed that their proposed framework offering user control paired with high explainability results in higher user satisfaction and acceptance. The integration of sliders for defining weights and mathematical breakdowns along with transparency panels investigated in this paper builds on those findings to build an explainable and transparent UI for user satisfaction and to meet the DSA requirements.

4 System Architecture and Implementation

The prototype is designed as a web application. The technical architecture consists of React, Vite for the build system, and vanilla CSS for layout and styling. Client-side math rendering is powered by KaTeX, a fast LaTeX math rendering library, integrated via a custom React wrapper. The system architecture is structured to support real-time user interaction.

The application state manages:

- The selected city (Munich vs. Vienna).
- The selected algorithm (MAUT vs. Constraint-based).
- The five attribute weights ($w_i \in [0, 10]$) for the MAUT mode.
- The five attribute limits for the Constraint Filtering mode.
- The global formula toggle (including a switch for the explanation board, showing the matching explanations).

In the dataset, which is stored in a static module "data.js", 20 hotels (10 for each city) are stored. The hotels represent a realistic and diverse range of accommodations, which range from low-budget hostels to luxury five-star hotels. Each hotel is uniquely defined by an identifier, name, city description, and five attributes:

(1) **Price per Night**: Ranging from $35 (Hostel Vienna West) to $550 (Mandarin Oriental Munich).
(2) **Distance to Center**: Distance from the historical city center (Stephansplatz in Vienna, Marienplatz in Munich). Ranging from 0.1 km (Stephansplatz) to 28.0 km (Airport Express Inn Munich).
(3) **Star Rating**: Ranging from 1 (basic tourist class) to 5 stars (luxury class).
(4) **Guest Review Score**: A continuous rating between 1.0 (worst) and 10.0 (excellent), representing aggregated guest satisfaction.
(5) **Public Transport Access Score**: A rating between 1.0 (poor) and 10.0 (excellent), representing proximity to subway stations, tram lines, and frequency of transit services.

The application calculates and updates the recommendations in real-time if a user modifies the slider weights or switches the algorithm. This renders a "calculate" button useless, improving user-friendliness. This is possible as the calculations on the fixed dataset are performed client-side.

5 Algorithmic Framework

To support the user-controlled interaction model, this section expains the mathematical models behind the algorithms in the recommendation engine.

5.1 Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT)

This algorithm evaluates each hotel in the chosen city by mapping the hotels attribute values to normalized utility scores and computing a weighted average. The process is performed in three steps:

5.1.1 Normalization of Attributes. As the attributes use incomparable units (e.g., USD, kilometers, stars), they must first be normalized to a standard scale $u(v) \in [0, 1]$, where 0 represents the worst possible utility and 1 represents the best possible utility.

The hotel attributes are split into cost metrics (where lower values are better, like price and distance) and benefit metrics (where higher values are better, like star ratings, guest review scores, and public transport access). The prototype implements two normalization strategies:

Relative Normalization: Used for cost metrics, as these values do not have clear upper and lower bounds. Therefore the items are compared to each other to define the thresholds. The normalized utility score $u(v)$ for an attribute value v of an item is calculated as:

$$u(v) = \frac{v_{\max} - v}{v_{\max} - v_{\min}} \tag{1}$$

where:

- v is the value of the hotel
- $v_{\min}$ and $v_{\max}$ are the minimum/maximum value of this attribute of all hotels in the selected city.

Absolute Normalization: This is used for benefit attributes (stars, rating, and public transport access) because these values have clear intrinsic values for the best and worst option (e.g. stars, guest review rating). In addition, normalizing these scores relatively could artificially exaggerate small differences (e.g. ranking a hotel with 8.5 rating as 0 utility if it is the lowest in the city). For stars (on a 1 to 5 scale), the utility is calculated as following:

$$u_{\text{stars}}(v) = \frac{v - 1}{4} \tag{2}$$

where v is the amount of stars of the hotel (from 1 to 5).

For the guest review rating scores and the public transport access score (both ranging on a 1 to 10 scale), the utility is calculated as following:

$$u_{\text{rating/transport}}(v) = \frac{v - 1}{9} \tag{3}$$

where v is the guest review score or public transport access score of the hotel (from 1 to 10).

5.1.2 Weighted Aggregation. After the initial normalized attribute scores are calculated, the utility scores are weighted and summed up. Let w_i be the user given weight to the attribute i (where $w_i \in [0, 10]$), and u_i be the prior calculated normalized utility score of the attribute i for the hotel item. The total utility U is calculated as:

$$U = \frac{\sum_{i=1}^{5} w_i \cdot u_i}{\sum_{i=1}^{5} w_i} \times 100\% \tag{4}$$

The calculated hotels are sorted and shown to the user in descending order.

5.2 Algorithm B: Constraint-Based Filtering

The Constraint-Based Filtering algorithm does not calculate a mathematical match score or rank the hotels. Instead, it checks if the set of hotels in the selected city comply to user set constraints. A hotel is in the result set (i.e. shown to the user) if and only if it satisfies the logical conjunction:

$$\text{Price} \leq P_{\max} \ \wedge \ \text{Distance} \leq D_{\max} \ \wedge \ \text{Stars} \geq S_{\min} \ \wedge \ \text{Rating} \geq R_{\min} \ \wedge \ \text{Transport} \geq T_{\min} \tag{5}$$

where:

- $P_{\max}$ is the maximum price threshold defined by the user.
- $D_{\max}$ is the maximum distance threshold defined by the user.
- $S_{\min}, R_{\min}, T_{\min}$ are the minimum stars, guest review rating, and public transport access scores thresholds defined by the user.

This filtering logic is non-compensatory — hotels are not compared to each other and relatively ranked. All hotels are simultaneously checked and excluded form the results if a single constraint is violated.

5.3 Comparison of Decision Logic

The two models are fundamentally different in their decision-making styles: The biggest difference is that MAUT will (due to its ranking nature) show results to the user that are completely irrelevant to the user, as it is more a sorting approach than a filtering approach. Constraint-filtering on the other hand does not show the aforementioned uninteresting results, but delivers a result set to the user, without giving information of how well an item fit the user's limits. The differences are summarized in Table 1.

Table 1. Comparison of the Implemented Algorithmic Paradigms

Dimension	Algorithm A: Weighted MAUT	Algorithm B: Constraint Filtering
Decision Logic	Compensatory (trade-offs allowed)	Non-compensatory (hard boundaries)
Output Type	Ranked list with continuous match scores	Unordered subset of matching items
User Controls	Relative importance weights ($w_i \in [0, 10]$)	Threshold limits ($P_{\max}$, $D_{\max}$, etc.)
Cognitive Style	Optimizing	Screening / Satisficing
DSA Compliance	Explains weights	Explains inclusion/exclusion criteria
Profiling Dependency	None (Profiling-free, real-time input)	None (Profiling-free, real-time input)

5.4 Quality Assurance and Algorithmic Verification

In order to ensure that the prototype is stable and mathematically correct, the code of the algorithm was built in a standalone module (`src/recommenderEngine.js`). There, for the verification a unit testing suite was appended (`run-tests.js`) consisting of exactly 60 different test cases (30 for Weighted MAUT and 30 for Constraint-Based Filtering) for edge cases and different input parameters patterns.

5.4.1 MAUT Logic Verification (30 Cases). The 30 MAUT test cases evaluate the logic of normalization and scoring:

- **Weight Boundary Conditions (5 cases)**: These tests verify that equal weights result in equal ranking results, zero weights do not result in division by zero and that no negative weights and no positive weights above 10 are possible.

- **Price relative cost normalization (5 cases)**: Confirms that the Relative Normalization works as expected: correct mapping of the lowest price to utility 1, highest price to utility 0, and values in between are correctly mapped proportionally. Additionally, it is also verified that no division by zero occurs when all items have the same price (algorithm could map them all to 0 as they all are the "worst" option). These tests are run for all other attributes:
- **Distance relative cost normalization (5 cases)**: Verifies that the closest hotel maps to the utility of 1.0, furthest hotel to the utility 0.0, and out-of-bound distances are clamped.
- **Absolute Normalizations (5 cases)**: Asserts that the absolute utility mappings for stars (1-to-5 scale) and guest ratings/public transport scores (1-to-10 scale) are correct.
- **Weight Dominance and Flipped Rankings (5 cases)**: These tests test that highly weighted attributes dominate the ranking and changing weights correctly changes the ranking order as expected.
- **Robustness and Sorting (5 cases)**: Verifies the edge cases for empty list inputs, null inputs, missing weight keys, strict descending sorting order of match scores, and correct formatting of breakdown scores.

5.4.2 Constraint-Based Filtering Verification (30 Cases). The 30 filtering test cases check the correctness of the logical conjunction:

- **Individual Attribute Constraints (25 cases)**: These tests check the price, location, amount of stars, guest review scores, and public transit access scores separately. For each category, there are 5 tests that verify exclusions and inclusions and that constraints may result in empty sets and in sets with all items included. Also, it is checked that null values lead to the usage of fallback values.

All test cases run with its own mock database and pass.

6 User Interface Design and Explainability

The prototype frontend interface was developed with the help of Gemini 3.5[1] in a modern style with smooth animations to create a premium and modern UI. The UI was also designed to feature maximal explainability for the users.

6.1 Landing Page for Algorithm Selection

Prior iterations showed detailed algorithm explanations before the selection, leading to overloading new users (see the previous cluttered layout in Figure 1). This issue was tackled by implementing a landing page which explains the available algorithms in simple words and prompts the user to select an algorithm. The user is shown 2 cards to select from:

- **Weighted MAUT Card**: Focuses on compensatory trade-offs. It explains that the user can rate attributes on a scale of 0 to 10, resulting in a ranked list of hotels with percentage match scores.
- **Constraint-Guided Card**: Focuses on strict non-compensatory boundaries. It explains that the user can set threshold limits to eliminate hotels violating any criteria.

Afterwards, the user may set the weights or constraints depending on their chosen algorithm and is shown a transparency board, which details the algorithm. This landing-page approach makes the prototype more transparent, as it explains the logic to all users in the beginning. The user may always return to the screen and change their choice by clicking on a "Switch Algorithm" button or change the algorithm directly via a switch on the main page.

6.1.1 Global Explanation Banners. At the top of the main page, there is a transparency board which changes the color and content based on the selected algorithm:

[1] https://deepmind.google/models/gemini/flash/

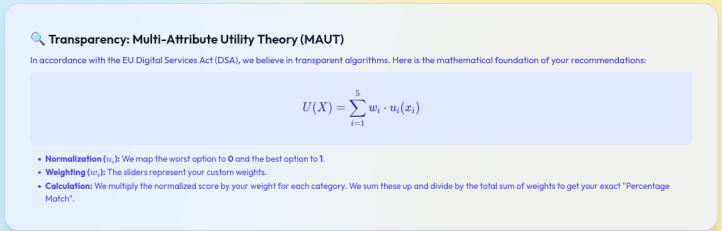

Fig. 1. Previous cluttered transparency board design that led to premature disclosure of complex information.

- In the **Weighted MAUT** mode, the banner has a blue color and explains the logic of the weighted MAUT algorithms and the sliders, see Figure 2.
- The **Constraint-Guided** mode banner is shown in green, explaining the logic behine the non-compensatory filtering approach, see Figure 3.

Both banners use simple language to not overwhelm users with the mathematical details. For interested users, the banners include a "Show Mathematical Details" button. When clicked, the banner renders the underlying formulas in LaTeX using KaTeX (Equation 4 for MAUT, and Equation 5 for Constraint-Guided Filtering) and the text is appended by an explanation of the variables.

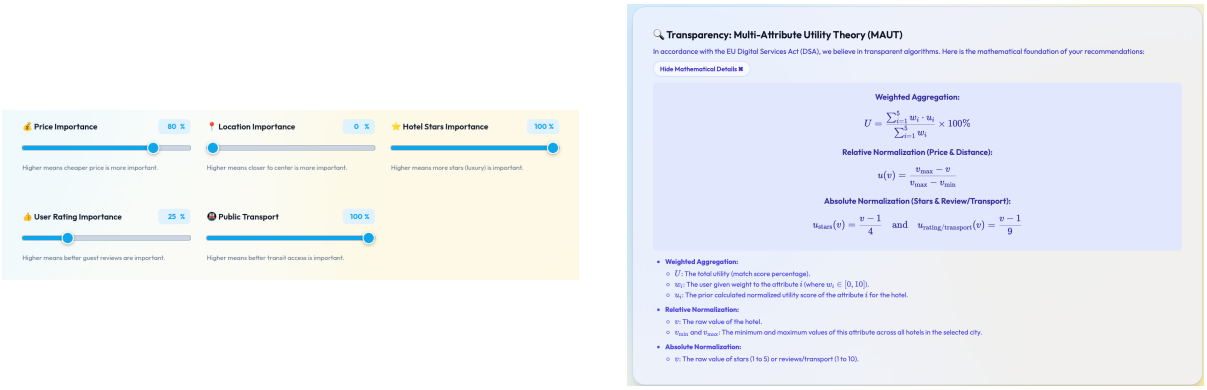

Fig. 2. Weighted MAUT user controls (left) and the dynamic blue explanation banner showing algorithmic details (right).

6.1.2 On-Demand Collapsible Math Breakdowns. In the Weighted MAUT mode, after the user has set the sliders, the user can inspect the calculated results. Each hotel card offers a "Show Mathematical Breakdown" button to understand how this particular score (match percentage) was calculated, (see Figure 4).

This feature makes sure that the prototype offers absolute transparency while not overloading not interested users. The user can understand why a hotel is ranked in its current position and can compare it to better or worse ranked hotels. By adjusting the sliders the user can also check the mathematical impact of the slider weights on the results. This eliminates the perception of this recommender system as a black box.

7 Future Work

While the presented prototype offers many transparency features and shows that they are feasible in the tourism domain, there is still research and further development required.

Fig. 3. Constraint-based filtering interface featuring threshold sliders and the dynamic green explanation banner.

Fig. 4. Collapsible step-by-step mathematical breakdown showing the exact utility calculations for a hotel.

7.1 The Hybrid Algorithmic Approach

The used approaches in this prototype have clear strengths and weaknesses:

- The weighted MAUT algorithm compares the different hotels to each other, but does not filter out irrelevant items. For instance, a user only interested in low-budget hotels will also be recommended a high priced hotel (with a low match percentage).
- The Constraint Based mode filters these irrelevant options out, but does not rank them, leading to higher user involvement, as they have to find the best-matching hotel for themself.

Therefore, a hybrid approach, which combines the clear strengths and eliminates the weaknesses is proposed for future work:

(1) **Stage 1: Constraint Filtering (Non-Compensatory screening)**: The system applies Constraint Filtering first to filter out any unacceptable hotels for the user (e.g., price > $250, or distance > 10 km). This assures that the candidates for further processing are limited to feasible options.
(2) **Stage 2: Weighted MAUT (Compensatory ranking)**: The system applies the user's preference weights to rank the preprocessed hotel candidates. This will ensure that the user will only see hotels in their domain (e.g. the aforementioned low-budget user won't see a luxury 5 star hotel with 8 percent match score).

This hybrid approach maintains full transparency, remains profiling-free and reduces the cognitive load. It represents a rational approach for human decision-making where first the unacceptable alternatives are filtered out and then the best fitting alternative is identified on the reduced set of options.

7.2 Proposed User Evaluation Protocol

To evaluate the user experience (UX) and the effectiveness of the explainability features, a controlled user study protocol is proposed.

7.2.1 Experimental Design. A within-subjects laboratory study with $N = 30$ participants is proposed. Each participant will perform travel planning tasks under three experimental conditions:

(1) **Condition 1 (Baseline - Black Box)**: A traditional recommender interface displaying hotel cards ranked by a hidden algorithm, with no sliders and no explanation panels.
(2) **Condition 2 (Weighted MAUT with On-Demand Explanations)**: The interactive slider-based interface with collapsible math breakdown panels.
(3) **Condition 3 (Constraint Filtering with Explanations)**: The constraint-based interface showing absolute threshold sliders.

The ordering of conditions will be counterbalanced to prevent learning effects. Following each condition, participants will complete standardized questionnaires to measure:

- **System Usability**: Measured via the System Usability Scale (SUS).
- **User Experience & Trust**: Measured using the ResQue (Recommender Systems Quality of Experience) framework, focusing on trust, perceived accuracy, control, and transparency.
- **Cognitive Load**: Measured via the NASA-TLX (Task Load Index) to evaluate if exposing mathematical details increases mental demand.

The results of this study will guide the refinement of the UI, helping to strike a balance between complete mathematical transparency and user cognitive ease.

8 Conclusion

The era of black-box recommender systems is facing both user-experience and regulatory challenges. This paper presented an interactive, transparent tourism recommender system prototype designed to address the compliance requirements of the EU Digital Services Act (DSA). By implementing both a compensatory Weighted Multi-Attribute Utility Theory (MAUT) model and a non-compensatory Constraint-Based Filtering model, the prototype provides users with direct, profiling-free control over recommendation logic. The weights are configured on an intuitive 0 to 10 scale to prevent percentage-summing confusion. We integrated tiered explainability features, combining simple plain-language summaries with on-demand, KaTeX-rendered mathematical breakdowns. Finally, we analyzed and resolved mathematical edge cases, including division-by-zero singularities and score overshooting, ensuring a stable and robust system. This architecture serves as a blueprint for developers seeking to combine rich user control, modern web aesthetics, and strict regulatory compliance in recommender system design.

Acknowledgments

The author would like to thank the teaching staff of the Lab Course: Projects in Recommender Systems (SS 2026) at the Chair of Connected Mobility, Technical University of Munich, for their guidance and feedback throughout the project.

References

[1] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering* 17, 6 (2005), 734–749. doi:10.1109/TKDE.2005.99

[2] Joan Borras, Antonio Moreno, and Aida Valls. 2014. Intelligent tourism recommender systems: A survey. *Expert Systems with Applications* 41, 16 (2014), 7370–7389. doi:10.1016/j.eswa.2014.06.007

[3] European Parliament and Council of the European Union. 2022. Regulation (EU) 2022/2065 of the European Parliament and of the Council of 19 October 2022 on a Single Market For Digital Services and amending Directive 2000/31/EC (Digital Services Act). http://data.europa.eu/eli/reg/2022/2065/oj.

[4] Mohamed Elyes Ben Haj Kbaier, Hela Masri, and Saoussen Krichen. 2017. A Personalized Hybrid Tourism Recommender System. In *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*. 244–250. doi:10.1109/AICCSA.2017.12

[5] Ralph Keeney, Howard Raiffa, and David Rajala. 1979. Decisions with Multiple Objectives: Preferences and Value Trade-Offs. *Systems, Man and Cybernetics, IEEE Transactions on* 9 (08 1979), 403 – 403. doi:10.1109/TSMC.1979.4310245

[6] Bart P. Knijnenburg, M. C. Willemsen, Z. Gantner, H. Soncu, and C. Newell. 2012. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction* 22, 4-5 (2012), 441–504. doi:10.1007/s11257-011-9118-4

[7] Sean M. McNee, John Riedl, and Joseph A. Konstan. 2006. Being accurate is not enough: How accuracy metrics have hurt recommender systems. (2006), 1097–1101. doi:10.1145/1125451.1125659

[8] Francesco Ricci. 2011. Travel recommender systems. In *Recommender Systems Handbook*. Springer, 527–556. doi:10.1007/978-0-387-85820-3_17

[9] Joy Lal Sarkar, Abhishek Majumder, Chhabi Rani Panigrahi, Sudipta Roy, and Bibudhendu Pati. 2023. Tourism recommendation system: a survey and future research directions. *Multimedia Tools and Applications* 82, 6 (2023), 8983–9027. doi:10.1007/s11042-022-12167-w

[10] Christian Schmitt, Dietmar Dengler, Michael Bauer, et al. 2002. The MAUT-Machine: An Adaptive Recommender System. In *Proceedings of the ABIS Workshop* (Hannover, Germany).

[11] Swearingen Sinha, Kirsten Medhurst, and Rashmi Sinha. 2001. Beyond Algorithms: An HCI Perspective on Recommender Systems. (09 2001).

[12] Nava Tintarev and Judith Masthoff. 2007. A Survey of Explanations in Recommender Systems. *IEEE 23rd International Conference on Data Engineering Workshop*, 801–810. doi:10.1109/ICDEW.2007.4401070

A Complete Hotel Database

Table 2 presents the complete dataset of 20 hotels used in the recommender system prototype, split between Vienna (v1–v10) and Munich (m1–m10).

B Author Information and Contribution Details

As a single-author project, I David Gallob worked entirely alone on this project. This involves all parts of this Practical Seminar from the development and testing of the prototype, creating and presenting all milestone meetings and final presentation as well as writing this final documentation.

C Generative AI Disclosure

This project utilized Generative AI tools (specifically Google DeepMind's Gemini 3.5 Flash) during development. The AI was employed as a pair-programming assistant to:

- Draft and optimize the user interface layouts and CSS styling rules.
- Debug and recommend JavaScript state handling best practices.
- Assist in structuring, polishing, and editing the LaTeX code of this scientific report.

All design decisions, algorithmic formulations, bug resolutions, and report text were audited, verified, and approved by the human author to ensure compliance with the scientific standards of the Technical University of Munich.

Table 2. Hotel Database Attributes

ID	Hotel Name	Price ($/night)	Distance (km)	Stars	Rating (/10)	Transport (/10)
		Vienna Hotels				
v1	Grand Imperial Vienna	350	0.5	5	9.6	9.5
v2	Boutique Hotel am Stephansplatz	180	0.1	4	9.2	10.0
v3	Danube Riverside Hotel	95	4.5	3	8.0	6.5
v4	Schönbrunn Palace Suites	290	6.0	5	9.8	8.0
v5	Hostel Vienna West	35	3.2	1	7.5	9.0
v6	Art Deco Hotel Prater	140	2.8	4	8.7	8.5
v7	Economy Inn Vienna	60	5.5	2	6.8	5.0
v8	Museum Quarter Retreat	210	1.2	4	9.4	9.0
v9	Green Oasis Hotel	110	7.5	3	8.9	4.0
v10	The Ritz-Carlton Vienna	450	0.8	5	9.7	10.0
		Munich Hotels				
m1	Bayerischer Hof	420	0.4	5	9.5	9.5
m2	Marienplatz Business Hotel	160	0.2	4	8.8	10.0
m3	Oktoberfest Lodge	85	2.5	2	7.2	8.0
m4	Englischer Garten Retreat	250	3.0	4	9.3	7.0
m5	Schwabing Youth Hostel	40	4.0	1	8.1	8.5
m6	Olympic Park Hotel	130	6.5	3	8.5	7.5
m7	Airport Express Inn	90	28.0	3	7.8	9.0
m8	Viktualienmarkt Boutique	190	0.6	4	9.1	9.5
m9	Isar River View	210	1.8	4	8.9	6.0
m10	Mandarin Oriental Munich	550	0.5	5	9.8	9.5